BEWT-CovExt: Evidence-Guided Multi-Agent Authoring and Validation of Web End-to-End Benchmark Extensions

Author Details Pending[1*]

[1*]Affiliation pending, Institution pending, Country pending.

Abstract

End-to-end web testing benchmarks support empirical comparison, but their value depends on whether test scenarios represent distinct, observable business behaviours and can be reproduced. We present BEWT-CovExt, an evidence-guided workflow and software framework for proposing and assessing extensions to the BEWT benchmark. The design combines coverage analysis, Gherkin scenario authoring, Java/Selenium implementation, bounded repair, explicit human review and provenance-aware validation. The study targets eight applications and registers forty candidate extensions, five per application. In an earlier audit, all forty Gherkin artifacts parsed, whereas the corresponding audited Java mappings were classified as wrong behaviour (22), partially implemented (14), or weak oracle (4). The final acceptance package records no evidenced human novelty-and-oracle approvals and no candidates meeting the complete scientific validation definition. On the tested acceptance revision, framework unit tests passed (23/23), while integration tests recorded 18 passes, one failure and one skip; all eight Maven test-source builds succeeded with test execution skipped. Historical execution summaries include eleven candidates reported to pass three reruns, but incomplete provenance and semantic review prevent treating these as validated benchmark additions. Fresh cloud environment and Selenium acceptance were not performed. These results illustrate why parsing, compilation and passing assertions are insufficient proxies for behavioural coverage, and motivate source-bound evidence and independent oracle review in AI-assisted benchmark construction.

Keywords: End-to-end testing, benchmark validity, test generation, multi-agent systems, test oracle, behavioural coverage, reproducibility

1 Introduction

Web applications coordinate user interfaces, persistent state and remote services. End-to-end (E2E) tests exercise these components through observable workflows, but their execution depends on environment state, browser behaviour, test data and the correctness of assertions. Benchmarks such as BEWT provide shared applications and tests for comparing E2E techniques [1]. A benchmark's usefulness also depends on the behaviours represented by its tests: an additional test file is not, by itself, evidence of a previously uncovered behaviour.

Large language models (LLMs) can assist test construction, and coverage-guided generation has shown promise in unit-test settings [2, 3]. Transferring that idea to web E2E testing raises distinct challenges. A plausible Gherkin scenario may describe unsupported behaviour; a compilable Selenium test may execute different steps; and a passing assertion may observe an incidental page element rather than the requested business outcome. These problems are instances of the test-oracle problem [4]. Agent workflows can combine model calls with tools and state [5, 6], but orchestration does not establish the validity of the resulting evidence.

We investigate an eight-application, forty-candidate target through four research questions concerning scenario validity, behavioural coverage, executability and workflow utility. Our contributions are (1) a multi-role, evidence-guided framework and its validation contracts; (2) a methodology linking baseline behaviours, scenario specifications, implementations and independent review; and (3) an empirical account of engineering checks and unresolved semantic, runtime and provenance failures. The contribution is a research method and a bounded evidence base, not a claim that forty validated tests were added. Section 2 situates the work; Sections 3–6 describe the questions, method, architecture and setup; Sections 7–11 report and interpret the evidence.

2 Background and Related Work

BEWT supplies applications and E2E test artifacts for research on web testing [1]. Selenium WebDriver automates browser interactions through a browser-specific driver [7]; Gherkin describes examples using structured steps that can make preconditions, actions and outcomes explicit [8]. Neither a readable scenario nor a successful browser command guarantees that an assertion captures the intended requirement.

CodaMosa uses language models to escape coverage plateaus in automated test generation [2], while TestPilot examines LLM-generated JavaScript unit tests [3]. These studies motivate generation and measurement but do not establish their effectiveness for multi-application, stateful web E2E benchmark expansion. ReAct interleaves reasoning and external actions [5]; BEWT-CovExt instead concerns a specific engineering workflow with explicit review and evidence boundaries. The oracle survey by Barr et al. explains why deciding whether an observed outcome is correct remains difficult [4]. We therefore treat a business oracle as a separately reviewable research object and avoid claiming novelty over all prior agent or testing systems.

3 Research Questions

RQ1: Scenario novelty and validity. How many candidates are valid, distinct from the baseline and approved by a reviewer? The intended measurements are parsing, behaviour-level comparison and reviewer decisions with reasons. Syntax is observable without proving novelty; absent authenticated review cannot be inferred from an implementation authorisation.

RQ2: Behavioural coverage extension. Which new behaviours or conditions are covered relative to the original benchmark? This requires a source-backed baseline inventory, a defined unit of behaviour, non-overlap assessment and validated post-condition evidence. The intended gain is the number of verified new units divided by the baseline inventory size; an unverified denominator or numerator makes the ratio unavailable, not zero.

RQ3: Executability and stability. Can the implementations compile, exercise the specified behaviour with a correct oracle and repeat after reset? Measurements include source-bound build and E2E outcomes, fixture and cleanup evidence, an initial execution and three additional clean-state reruns. Runs from different revisions or environments cannot be silently pooled.

RQ4: Workflow utility and operational cost. What model calls, repairs, elapsed time, human interventions and provider usage are observable? Actual provider usage, estimated charges and missing telemetry must be separated. No comparison with manual or single-prompt methods is warranted without a controlled comparison and measured effort.

4 Research Methodology

The study adopts the eight applications identified by the project contract and targets five candidate scenarios per application. Proposal counts and version labels are planning inputs, not proof of pinned, runnable environments. The project records a mismatch between business-case entries and extracted scenario headers, so the nominal baseline total is not a defensible behavioural coverage denominator. The protocol retains existing benchmark source, records the proposed gap and its evidence, and compares each candidate against the relevant baseline rather than inferring novelty from wording alone.

A candidate links a Gherkin specification to a Java/Selenium implementation and a reviewable business oracle. Review should establish that the behaviour is supported by independent requirements or application evidence, is distinct from the baseline, and has a meaningful expected outcome. Every essential Given, When and Then must map to the immutable Java revision and its assertions. A negative test requires a reasoned expected outcome; merely observing the current application response does not establish correctness.

The gates in Table 1 organize the study. In the project contract, *proposed*, *accepted for implementation*, *implemented*, *compiled*, *executed* and *validated* refer to different evidence states. A validated case requires an approved novelty and oracle decision, traceability to source and implementation, a correct observed E2E outcome, and

three additional clean-state reruns with fixture, runtime and cleanup provenance. A technical pass cannot substitute for a missing review or semantic mapping.

Table 1 Research gates and minimum evidence.

Gate	Required evidence
G0	Pinned environment, readiness, meaningful baseline test, fixture, reset and owned-resource cleanup.
G1	Source-backed inventory, behavioural comparison and independent novelty/oracle decisions.
G2	State, role contracts, checkpoints, review, bounded tools and authentic model provenance.
G3	Source-bound implementation, correct oracle, clean execution and three additional reruns.
G4	Reconciled metrics, reproducible package and required human sign-off.

5 Framework Architecture

The TypeScript framework uses LangGraph state and four logical roles: coverage analysis, Gherkin generation, Selenium implementation, and validation/repair. These are responsibilities within the workflow, not evidence of four independently evaluated models. State contracts include application and source identity, candidate revision, review decisions, artifacts, status and repair count. SQLite checkpointing supports a paused review and subsequent resume in the implemented path. Review may accept for implementation, request revision or reject; this decision is distinct from the later scientific approval of novelty and oracle.

Figure 1 summarizes the implemented and intended evidence flow. The application manifest restricts tool use; fixed dispatch and path checks constrain operations. A bounded repair counter permits at most two attempts, and the production path restricts repair to classified generated-test defects. Artifact registration and invocation identifiers support provenance. These mechanisms are implementation and contract claims: they do not establish that every registered case came from a live model or that all intended runtime paths succeeded. The final integration failure further limits any blanket reliability claim.

6 Experimental Setup

Table 2 lists the project proposal's benchmark labels and nominal original business-case counts. Labels are not substituted for immutable image digests or source commits. In particular, the proposal label ExpressCart 1.19 and observed upstream package version 1.1.19 should not be treated as proof of identical source. The framework uses Node.js/TypeScript and LangGraph; application test harnesses use Maven and Java/Selenium. Docker-based environments, isolated fixtures and clean resets form part of the intended protocol, including a cloud deployment where separately evidenced. Their presence in the design does not establish a fresh final execution.

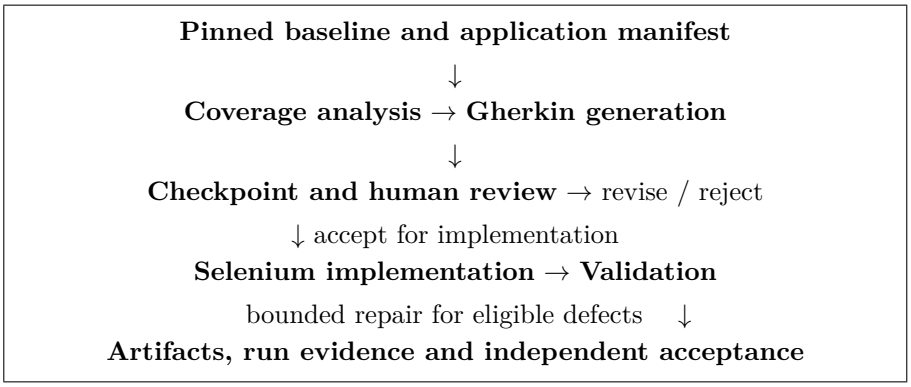


Fig. 1 Evidence flow through the multi-role workflow. Review and scientific acceptance are distinct; the diagram depicts contracts and code paths, not an assertion that all paths were successfully exercised.

The manuscript evidence snapshot is repository commit 0c7f1e35fc0ad82ccf37df82c86b08863e83b9cf [9]. The final acceptance commands were actually run on 55a3a3a6c3ea9fbffa72e773aaab57496e84a93f, before the closure record was integrated. The earlier detailed semantic audit concerns its recorded historical R2 revisions and baseline; subsequent integration of selected revisions prevents treating that distribution as a new audit of all current Java files. The earlier ExpressCart feasibility study is historical and separate: it reported two final E2E passes among four implementations, with repeats for those two; it did not establish program-wide novelty or measured behavioural gain.

Table 2 Application inventory and earlier R2 semantic audit. Proposal labels and baseline counts are not verified coverage denominators. Each application has five registered candidates. W/P/O denote wrong behaviour, partial implementation and weak oracle, respectively.

Application	Label	Baseline	W	P	O
Bludit	2.3.4	23	1	3	1
Claroline	1.11.10	40	4	1	0
ExpressCart	1.19	31	4	0	1
Joomla	3.10.11	35	2	2	1
Kanboard	1.2.15	30	1	3	1
MantisBT	1.2.0	37	5	0	0
MediaWiki	1.40.0	27	4	1	0
PrestaShop	1.6.1.23	35	1	4	0
Total	–	258	22	14	4

7 Results

7.1 RQ1: Scenario generation and validity

The earlier recomputed audit records valid Gherkin syntax for 40/40 registered candidates. Its semantic mapping audit, however, classified the corresponding R2 implementations as 22 wrong behaviour, 14 partially implemented and four weak oracle; no mapping was judged fully aligned. These are historical audit classifications, not a claim that all later revisions retain the same defects. The final acceptance package evidences zero signed human novelty-and-oracle approvals for the forty candidates. Thus neither the valid novel proposal rate nor an approved coverage extension can be established from parsing and implementation authorisations alone.

7.2 RQ2: Behavioural coverage extension

The inventory contains forty registered candidates, five for each application. Candidate presence describes proposed scope, not verified new coverage. The baseline behaviour denominator and the count of verified new, non-overlapping units remain unestablished. Consequently no numerical behavioural coverage gain is reported. In particular, adding forty candidate files to the proposal's 258 business-case entries would conflate different units and acceptance states.

7.3 RQ3: Executability and reproducibility

Table 3 separates the final engineering checks from research validation. Maven returned BUILD SUCCESS in all eight harnesses using `-DskipTests test`; this establishes test-source build success at module level, not execution or correctness of forty individual scenarios. The acceptance run did not perform fresh cloud/Docker G0, Selenium G3 or clean-state repeats. The final package records 0/40 cases satisfying the complete VALIDATED definition.

Historical G3 accounting preserves 79 canonical run identifiers: 13 candidates reportedly passed initial execution and 11 reportedly passed three reruns. These are observations of recorded assertions, not source-bound final acceptance or independent human approval. The accounting also identified 84 run-record files and 85 JUnit artifacts, including supplementary records and an unpaired XML. At least one cleanup record did not verify zero residual resources. These discrepancies are not an invitation to add all artifacts into a new success denominator.

The distinction is concrete. In the historical BL-GAP-002 R2 example, the scenario requires selecting a cover image, saving, reloading and displaying it publicly. The audited Java creates content and checks a title and container but neither selects nor verifies the image. Even a recorded passing rerun would therefore fail to demonstrate the specified oracle. This example illustrates the mapping defect, not a new execution of the latest revision.

Table 3 Final acceptance checks on the tested source revision. These checks have different units and must not be combined as a single success rate.

Check	Outcome
Framework build	PASS
Framework unit tests	23/23 PASS
Framework integration tests	18 PASS, 1 FAIL, 1 SKIP
Maven test-source builds, tests skipped	8/8 BUILD SUCCESS
Metrics negative controls	29/29 PASS
G3 accounting tests	3/3 PASS
SonarCloud quality gate	FAIL
Fresh G0, G3 and three reruns	NOT VERIFIED
Scientifically validated candidates	0/40

7.4 RQ4: Workflow utility and operational cost

The framework build and unit tests support the presence of functioning components under their test conditions. The integration suite's failed expectation for one Java repair call (expected one, observed zero), and one skipped replay test, constrain the claim of end-to-end reliability. Historical model records include failed calls and a later authorised request that failed on a provider schema error. They do not establish live model generation of all forty artifacts. Provider usage and actual cost are unknown where telemetry is absent; human time savings and comparative advantage were not measured.

7.5 Cross-application comparison

Table 2 shows heterogeneous historical semantic failure modes, with five audited cases per application. The same numerical denominator does not make the applications independent replicates of a common population: cases share tools, authoring processes and application-specific constraints. All eight modules compiled in the final build check, but this does not establish eight working runtime environments. Zero evidenced final approvals and zero validated cases apply to the registered set, not to the correctness of every conceivable future revision.

8 Discussion

The results expose a gap between artifact production and benchmark validity. A parser assesses grammatical structure; a compiler checks language and dependency constraints; a test runner evaluates assertions present in the program. None of these alone determines whether the program exercises the scenario's business condition. Weak assertions can pass because the page exists or a generic element is visible even when a crucial action or postcondition is absent. This is consistent with the broader oracle problem [4] and is particularly consequential when generated tests are used to evaluate other testing tools.

A useful benchmark extension must preserve the chain from a documented gap to a reviewable scenario, an implementation of its essential steps, a justified oracle and a

reproducible observed outcome. The failure of any link changes what a result means. An implementation authorisation can support further engineering without proving novelty; a successful rerun may support repeatability of an assertion without establishing its semantic relevance. The framework makes such distinctions expressible, while the available evidence shows that expressibility did not ensure their satisfaction in practice.

The study also highlights provenance as a measurement requirement. A historical run without sufficient source hashes, runtime identity, reset and cleanup records may be valuable diagnostic evidence, but it cannot verify a current revision. The final failed integration test and quality gate are operational constraints, not direct measurements of all candidate scenarios. Future claims about productivity require a defined comparison, complete call accounting and human effort measurement.

9 Threats to Validity

Construct and measurement validity. Candidate count, parsing and observed passes are imperfect proxies for novelty, behaviour and correctness. The baseline inventory contains mismatched units and has no verified behavioural denominator. The semantic audit is revision-specific and review decisions lack the required trusted evidence.

Internal validity. Environment state, test order, fixture dependencies and missing reset evidence can affect outcomes. Failure classification may confuse infrastructure problems with generated-test defects. Unbound historical runs cannot establish the effect of a later source revision.

External and selection validity. Eight selected benchmark applications and the observed model attempts do not represent all web systems, models or workflows. Selection of five candidates per application can hide rejected or never-generated proposals; the forty registered candidates are not forty independently sampled applications.

Conclusion validity and reviewer bias. Three reruns provide only limited stability evidence, and dependent cases should not be treated as independent samples. Reviewers aware of artifact origin may be biased; missing signed decisions prevent estimating agreement or approval rates. Null measurements are reported as unavailable rather than converted to zeros. No statistical effectiveness or comparative cost claim follows from these data.

10 Limitations and Future Work

Independent human review of novelty and oracle evidence remains necessary for each candidate. The audited semantic defects require source-specific correction and rereview; subsequent revisions should be evaluated on their own merits. Fresh G0 and G3 runs require pinned environments, independently verified reset and cleanup, artifact hashes, correct assertions and three additional clean-state reruns. A reconciled baseline inventory is needed before reporting behavioural coverage gain.

The failed framework integration assertion and SonarCloud gate require investigation; the latter reported new-code duplication and security and reliability ratings below its configured thresholds. Provider-side usage telemetry, elapsed time and

reviewer effort must be collected before making cost or productivity claims. The closure report counts ten previously recorded blocker categories plus two additional acceptance failures; this is an issue-accounting convention, not a count of independent scientific defects. These are proposed next steps, not completed experiments.

11 Conclusion

BEWT-CovExt contributes a structured multi-role workflow, candidate artifacts and an evidence-oriented protocol for extending a web E2E benchmark. The study registered forty candidates across eight applications and established several engineering checks, but the available acceptance evidence supports no scientifically validated extension and no quantified behavioural coverage gain. Historical parsing and execution results remain useful observations when tied to their actual revisions and limitations. The principal methodological finding is that trustworthy benchmark growth requires independent business-oracle review and reproducible, source-bound evidence in addition to automated generation and execution.

Declarations. **Funding:** Editorial confirmation required. **Competing interests:** Author declarations required. **Author contributions:** To be supplied by the authors. **Ethics approval and consent:** Applicability and institutional requirements require author confirmation. **Data and code availability:** The public research repository and evidence paths are cited as an artifact [9]; external runtime resources and credentials are not included. Availability and licensing of third-party benchmark assets must be confirmed before submission.

Appendix A Evidence and Reproduction Notes

The accompanying evidence matrix identifies the final acceptance report, per-case semantic audit and raw execution accounting. The companion evidence matrix maps manuscript claims to these files and their temporal boundaries. The framework's recorded acceptance commands include `npm run build`, `npm run test:unit` and `npm run test:integration`; Maven used `-DskipTests test`. Running them again is a new observation, not reproduction of cloud or Selenium acceptance. Environment provisioning, secrets, reset ownership and human review must be resolved before any new live experiment.

References

[1] Olianas, D., Leotta, M., Ricca, F.: BEWT: Extended benchmarking for end-to-end web testing. Journal of Systems and Software **237**, 112849 (2026) https://doi.org/10.1016/j.jss.2026.112849

[2] Lemieux, C., Inala, J.P., Lahiri, S.K., Sen, S.: CodaMosa: Escaping coverage plateaus in test generation with pre-trained large language models. In: Proceedings of the 45th International Conference on Software Engineering, pp. 919–931 (2023). https://doi.org/10.1109/ICSE48619.2023.00085

[3] Schäfer, M., Nadi, S., Eghbali, A., Tip, F.: An empirical evaluation of using large language models for automated unit test generation. IEEE Transactions on Software Engineering **50**(1), 85–105 (2024) https://doi.org/10.1109/TSE.2023.3334955

[4] Barr, E.T., Harman, M., McMinn, P., Shahbaz, M., Yoo, S.: The oracle problem in software testing: A survey. IEEE Transactions on Software Engineering **41**(5), 507–525 (2015) https://doi.org/10.1109/TSE.2014.2372785

[5] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y.: ReAct: Synergizing reasoning and acting in language models. In: International Conference on Learning Representations (2023). https://arxiv.org/abs/2210.03629

[6] LangChain: LangGraph Overview. https://docs.langchain.com/oss/python/langgraph/overview. Accessed 13 November 2026

[7] Selenium Project: WebDriver Documentation. https://www.selenium.dev/documentation/webdriver/. Accessed 13 November 2026

[8] Cucumber: Gherkin Reference. https://cucumber.io/docs/gherkin/reference/. Accessed 13 November 2026

[9] BEWT-CovExt: Research Repository and Acceptance Evidence. https://github.com/NgaiLong49423/BEWT-CovExt/tree/0c7f1e35fc0ad82ccf37df82c86b08863e83b9cf. Research snapshot, commit 0c7f1e35fc0ad82ccf37df82c86b08863e83b9cf; accessed 13 November 2026